



ROS2

Guía para Simulación mediante ROS2

Código: 215511699
Alumno: Marquez Pinto Martin Alejandro
Carrera: Ingeniería Robótica (INRO)
Fecha de Inicio: 01/Junio/2026

Resumen: En este documento se añadirán todos los avances para llevar un modelado 3D CAD desde Fusion360 hasta ROS2, simulando el modelado mediante Gazebo y visualizando su movimientos mediante Rviz2, gracias a WSL y la instalación de Ubuntu 24.04

Resumen Detallado

Índice general

Resumen Detallado	1
1. Fusion360 a ROS2	3
2. Instalación del subsistema WSL 2 (Ubuntu) e integración de ROS 2 Jazzy y Gazebo Harmonic	4
3. Exportar modelado a .urdf o .xacro	7
4. Ejemplo practico con carrito_prueba_description	9
5. Lanzar Rviz en segundo plano mientras se lanza Gazebo	14
Referencias e Hipervínculos	16

Capítulo 1

Fusion360 a ROS2

1. **Base:** La "base."º "ground", el torso al que se le unirán todos los componentes debe ser llamado `base_link` y estar fijado/anclado/ground.
2. **Solo minúsculas (snake_case):** Al nombrar los componentes, articulaciones y el proyecto final, usa solo letras minúsculas y guines bajos, evitando espacios (ej. `pendulo_base`, `eslabon_uno`).
3. **Estructura de Componentes:** Cada pieza móvil debe ser un Componente Independiente, no debe estar enlazado con nada mas del proyecto (simetrías, copias, etc).
4. **Antes de crear los Joints:** Mover las piezas justo a su posición final, que la opción de Joint no los mueva por su cuenta, importante para el reconocimiento de su origen.
5. **Asignación de Joint:** Define correctamente los limites de movimiento en Fusion, ya que ROS leerá esos limites matemáticos, las Referencias preferentemente en medio de los cilindros, cuidar que todo en el apartado de `.Alineación` este en 0° (Angulo, desfases[x,y,z]).
6. **Renombrar Uniones (Relaciones):** Cuando creas una Unión/Joint, se generara con el nombre de Revolución X", renombrar sin acentos ni espacios, por ejemplo `joint1/joint_1/llanta_delantera_izq`.
7. **Copiar Componentes (Complemento Paso 3):** Si se copean componentes, seleccionar el archivo principal (carpeta padre), hacer click derecho y "pegar nuevo", de esta manera los componentes se pegaran en la raíz, todos los componentes del proyecto serán "hermanos", sin ningún tipo de derivación entre ellos.

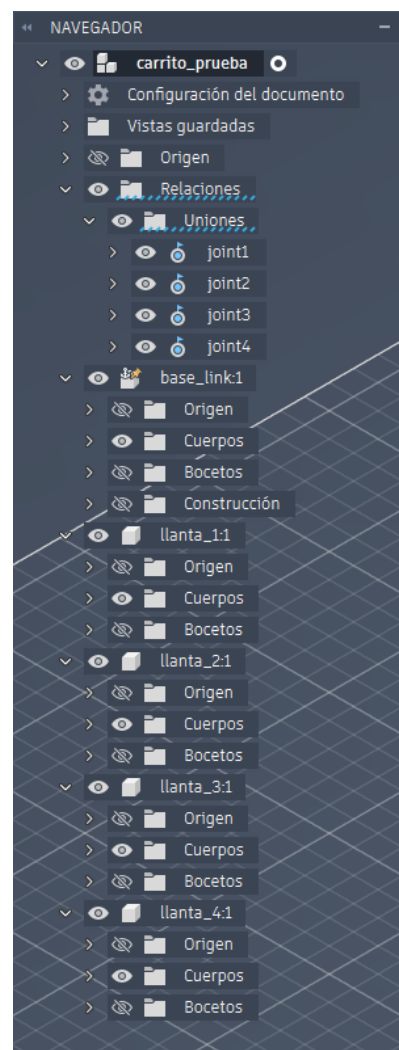


Figura 1.1: Árbol de trabajo Fusion360

Capítulo 2

Instalación del subsistema WSL 2 (Ubuntu) e integración de ROS 2 Jazzy y Gazebo Harmonic

Primero abrimos una terminal de PowerShell como administrador, una vez abierta vamos a continuar con los siguientes pasos:

1. **Instalar WSL y Ubuntu 24.04:** Una vez abierta la terminal, ejecutamos el siguiente comando: `wsl -install -d Ubuntu-24.04`, reiniciamos, abrimos Ubuntu 24.04 mediante la tecla Windows y escribimos Ubuntu en el buscador, si es que no aparece, volvemos a ejecutar el comando anterior.
2. **Configurar Usuario:** Una vez abierta la terminal de Ubuntu, escribimos nuestro nombre de usuario (minúsculas y sin espacios), este nos pedirá una contraseña, importante, no se verán asteriscos ni letras por seguridad, pero sí se está escribiendo. Presiona Enter cuando termines.
3. **Instalación del Núcleo (ROS 2 Jazzy):** Configuramos el sistema al Idioma Local

```
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Después habilitamos los repositorios de Ubuntu

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

Agregamos las llaves criptograficas de ROS 2

```
sudo apt update && sudo apt install curl -y
sudo curl -sSL
→ https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
→ -o /usr/share/keyrings/ros-archive-keyring.gpg
```

Añadimos el repositorio a la lista de fuentes

```
echo "deb [arch=$(dpkg --print-architecture)
→ signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
→ http://packages.ros.org/ros2/ubuntu $(. /etc/os-release &&
→ echo $UBUNTU_CODENAME) main" | sudo tee
→ /etc/apt/sources.list.d/ros2.list > /dev/null
```

Instalamos ROS 2 Jazzy (Versión Completa)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ros-jazzy-desktop
```

4. **Herramientas de Compilación y Simulación:** ROS2 necesita herramientas extra para compilar código, y nosotros necesitamos Gazebo para simular físicas.

Instalamos Colcon y dependencias de desarrollo

```
sudo apt install -y python3-colcon-common-extensions
→ ros-dev-tools
```

Instalar el puente de Gazebo Harmonic y la GUI de RViz

```
sudo apt install -y ros-jazzy-ros-gz
→ ros-jazzy-joint-state-publisher-gui
```

Automatizamos el Sourcing (Para no tener que escribir el comando de activación cada vez que abrimos la terminal, lo inyectamos directamente en el archivo de arranque **.bashrc**):

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

5. **Creacion del Espacio de Trabajo (Workspace):** Aquí es donde vivirá el código, modelos 3D y simulaciones.

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws
```

Si es que ya tenemos un modelo exportado y dentro de la carpeta src, hacemos la primera compilación.

```
cd ~/ros2_ws  
colcon build --symlink-install
```

(Opcional) Si queremos que el espacio de trabajo local cargue automáticamente al abrir una nueva terminal

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

6. **Lanzamiento:** Si todo esta correcto, abrimos la terminal e iniciamos los simuladores

```
ros2 launch pendulo_prueba_description gazebo.launch.py  
o  
ros2 launch pendulo_prueba_description display.launch.py
```

Extra:

```
sudo apt install ros-jazzy-gz-ros2-control
```

Sirve para instalar el puente de comunicación entre la arquitectura robótica ros2_control y el simulador de físicas Gazebo. Este paquete habilita un plugin de sistema que crea un gestor de controladores virtual en Gazebo, permitiéndote probar, mover y testear los motores y articulaciones de tu modelo digital exactamente igual que si fuera un robot real.

Versiones esperadas despues de la instalacion:

Gazebo Sim Server v8.11.0

gz_sim.launch.py

ros_gz_bridge

ros_gz_sim

Capítulo 3

Exportar modelado a .urdf o .xacro

■ Fase 1:

1. Descargar el script del repositorio de [runtimerobotics](#)
2. Añadir el script a Fusion360 mediante la pestaña de Utilidades - Complementos.
3. Darle click al boton runrepresentado con un triangulo horizontal, si el modelado fue correcto, este creara/exportara una carpeta con la terminación "_description" en la carpeta destino.
4. Si el modelado fue exportado sin texturas, entramos a la carpeta generada, subcarpeta urdf y eliminamos la carpeta vacía "_textures", ya que nos puede generar problemas estando vacía.

■ Fase 1.5 (VirtualBox):

1. Mandar la carpeta exportada a la carpeta compartida de Windows/Ubuntu.
2. Iniciar VB, abrir el gestor de archivos y copiar la carpeta "_nombre_carpeta_description" de la unidad compartida "sf...".
3. Pegar la carpeta dentro del WorkSpace de ROS 2, la ruta seria:
`carpeta_personal/ros2_ws/src/`

■ Fase 2 (Modificar archivo .xacro):

1. Generar un eslabón fantasma para que funcione como verdadero punto de anclaje al mundo, dejando que "base_link" conserve su masa e inercia sin que ROS se queje:
2. Vamos a la ruta `carpeta_personal/ros2_ws/src/` y abrimos el archivo "nombre_carpeta.xacro".
3. Buscamos al principio la etiqueta de "base_link".
4. Buscamos la linea `<link name="base_link">`

5. Arriba de la linea anterior, vamos a añadir el siguiente código:

```
<link name="base_footprint"/>

<joint name="base_footprint_joint" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
</joint>
```

■ **Fase 3:**

Abrir la terminal (Ctrl + Alt + T) y ejecutar la siguiente secuencia exacta (asumiendo que todo está perfecto, compilará en 3 segundos):

1. `cd /ros2_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 launch nombre_carpeta_description display.launch.py`

Para lanzar Gazebo disparar el siguiente launch:

5. `ros2 launch nombre_carpeta_description gazebo.launch.py`

■ **Fase 4 (Configuración para Rviz2):**

Una vez abierto Rviz2, buscar la opción de Fixed Frame en la parte superior izquierda y cambiar de “base_link” a “base_footprint” para cambiar el anclaje.

Capítulo 4

Ejemplo practico con carrito_prueba_description

Para esta prueba practica de simulación, haremos los siguientes movimientos en múltiples archivos, así como la creación del archivo .yaml

1. **Fase 1:** Agregamos el siguiente código a `urdf/carrito_prueba.ros2control`

- **Código Original:**

```
<?xml version="1.0" ?>
<robot name="carrito_prueba"
  ↪  xmlns:xacro="http://www.ros.org/wiki/xacro" >

</robot>
```

- **Código Nuevo:**

```
<?xml version="1.0"?>

<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <ros2_control name="GazeboSystem" type="system">

    <hardware>
      <plugin>gz_ros2_control/GazeboSimSystem</plugin>
    </hardware>

    <joint name="joint1">
```

```

        <command_interface name="velocity"/>
        <state_interface name="position"/>
        <state_interface name="velocity"/>
    </joint>

    <joint name="joint2">
        <command_interface name="velocity"/>
        <state_interface name="position"/>
        <state_interface name="velocity"/>
    </joint>

    <joint name="joint3">
        <command_interface name="velocity"/>
        <state_interface name="position"/>
        <state_interface name="velocity"/>
    </joint>

    <joint name="joint4">
        <command_interface name="velocity"/>
        <state_interface name="position"/>
        <state_interface name="velocity"/>
    </joint>

</ros2_control>

</robot>

```

2. **Fase 2:** Creamos el archivo .yaml manualmente o con el comando
`nano ~ /ros2_ws/src/carrito_prueba_description/config/ros2_controller.yaml`
 en la terminal.

■ **Código Nuevo:**

```

1      controller_manager:
2        ros__parameters:
3
4          update_rate: 100
5
6          joint_state_broadcaster:
7            type:
8              ↪ joint_state_broadcaster/JointStateBroadcaster
9
10         velocity_controller:
11           type:
12             ↪ velocity_controllers/JointGroupVelocityController

```

```

11
12
13     velocity_controller:
14         ros__parameters:
15
16         joints:
17             - joint1
18             - joint2
19             - joint3
20             - joint4
21
22         interface_name: velocity

```

3. **Fase 3:** Agregamos las siguientes líneas de código al archivo en la ruta `ros2_ws/src/carrito_prueba_description/urdf/carrito_prueba.gazebo`. Justo arriba de la última línea `</robot>`.

■ **Código Nuevo:**

```

<gazebo>
  <plugin
    filename="libgz_ros2_control-system.so"

    ↪ name="gz_ros2_control::GazeboSimROS2ControlPlugin">

    <parameters>
      $(find
        ↪ carrito_prueba_description)/config/ros2_controller.yaml
    </parameters>

  </plugin>
</gazebo>

```

4. **Fase 4:** Dentro del archivo:
`ros2_ws/src/carrito_prueba_description/launch/gazebo.launch.py`.
 Añadimos:
`joint_state_broadcaster_spawner` y `velocity_controller_spawner` después de `ros_gz_bridge` (Casi al final).
 También los agregamos a `LaunchDescription`.

■ **Código Original:**

```

ros_gz_bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',

```

```

        parameters=[{'config_file': ros_gz_bridge_config}],
        output='screen'
    )

    return LaunchDescription([
        gazebo,
        spawn_robot,
        ros_gz_bridge,
        robot_state_publisher,
    ])

```

■ Código Nuevo:

```

ros_gz_bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    parameters=[{'config_file': ros_gz_bridge_config}],
    output='screen'
)

joint_state_broadcaster_spawner = Node(
    package='controller_manager',
    executable='spawner',
    arguments=[
        'joint_state_broadcaster',
        '--param-file',
        os.path.join(
            pkg_ros_gz_rbot,
            'config',
            'ros2_controller.yaml'
        )
    ],
    output='screen'
)

velocity_controller_spawner = Node(
    package='controller_manager',
    executable='spawner',
    arguments=[
        'velocity_controller',
        '--param-file',
        os.path.join(
            pkg_ros_gz_rbot,
            'config',
            'ros2_controller.yaml'
        )
    ],
    output='screen'
)

```

```

        )
    ],
    output='screen'
)

return LaunchDescription([
    gazebo,
    spawn_robot,
    ros_gz_bridge,
    robot_state_publisher,
    joint_state_broadcaster_spawner,
    velocity_controller_spawner,
    rviz_node,
])

```

5. Fase 5: Compilación

- a) `~cd /ros2_ws`
- b) `colcon build --packages-select carrito_prueba_description`
- c) `source install/setup.bash`
- d) `ros2 launch carrito_prueba_description gazebo.launch.py`

Abrimos otra terminal Ubuntu

- e) `source /ros2_ws/install/setup.bash`
- f) `ros2 control list_controllers`

Lo importante es verificar si `joint_state_broadcaster` y `velocity_controller` estan activos, si es así, ejecuta.

- g) `ros2 topic pub --rate 10 /velocity_controller/commands std_msgs/msg/Float64MultiArray "data: [2.0, -2.0, 2.0, -2.0]"`

`ros2 topic pub` es un nodo temporal creado por nuestra terminal para comunicarse con el controlador `velocity_controller` mediante el tópico `\velocity_controller\commands`

- h) Si todo fue configurado correctamente, deberíamos ver como el carrito se mueve dentro del simulador Gazebo.

Capítulo 5

Lanzar Rviz en segundo plano mientras se lanza Gazebo

Cuando se lanza Gazebo mediante `gazebo.launch.py`, llamamos al nodo `robot_state_publisher`, el cual es el encargado de leer cómo giran las llantas en Gazebo y calcular el movimiento para ROS.

Si nosotros lanzamos Rviz mediante `display.launch.py` en otra terminal, este archivo de lanzamiento viene configurado por defecto con un nodo llamado `joint_state_publisher_gui`. Entonces tenemos un conflicto, ya que Rviz no está leyendo lo que Gazebo está mandando, no están enlazados de esta manera.

La solución a esto debería simplemente abrir una nueva terminal limpia y lanzar **rviz2**, asegurando que Rviz abre en un mundo completamente vacío, como ROS es el encargado de leer la matemática y Rviz solamente la refleja, esto debería visualizar el movimiento, configurando lo siguiente para configurar el mundo vacío:

1. En Fixed Frame, cambia la palabra map por base_link o odom.
2. Abajo a la izquierda, dale al botón Add (Añadir), busca la opción RobotModel y dale OK.
3. Vuelve a darle al botón Add, busca la opción TF (para ver los ejes de las llantas) y dale OK.

Aunque puede que esto no funcione ya que posiblemente se vean solamente los ejes, pero no encuentre los archivos 3D (stl) para dibujar el modelado.

Entonces posiblemente ocurra por el siguiente comportamiento de ROS2:

1. **Cuando usas display.launch.py:** El script de lanzamiento lee el archivo URDF directamente desde el disco duro de tu computadora como un archivo de texto local y se lo pasa a RViz. Como RViz lo lee de manera local, sabe perfectamente dónde ir a buscar la carpeta meshes para dibujar el diseño en la pantalla.

2. **Cuando abres rviz2 limpio desde la terminal:** RViz no está leyendo ningún archivo de tu disco duro; está escuchando de forma remota el tópico `/robot_description` que publica Gazebo. En algunas versiones de ROS 2 (como Jazzy o Humble), el motor gráfico de RViz (Ogre) tiene un detalle técnico: a veces no logra resolver correctamente las rutas del tipo `package://` cuando el modelo le llega de golpe como un texto plano a través de un tópico de la red, perdiendo la referencia de los archivos `.stl`

Para solucionar esto, podemos lanzar Rviz2 mientras abrimos Gazebo, inyectando a Rviz directamente dentro del archivo `gazebo.launch.py`. Compartiendo el mismo contexto, y el modelado 3D se cargará perfectamente sin errores y completamente sincronizado.

Para ello abrimos nuestro archivo de lanzamiento de Gazebo:

```
~nano \ros2_ws\src\carrito_prueba_description\launch\gazebo.launch.py
```

Nos desplazamos hacia abajo y justo antes de `return LaunchDescription`, añadimos el nodo encargado de arrancar RViz de forma nativa:

```
# 1. Creamos la ruta hacia el archivo de configuración existente del
→ coche
rviz_config_file = os.path.join(pkg_ros_gz_rbot, 'config',
→ 'display.rviz')

# 2. Configuramos el nodo corrigiendo el typo y agregando el argumento
→ de carga
rviz_node = Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2',
    output='screen'
)
```

Y añadimos el nodo `rviz_node` al final de la lista de retorno `LaunchDescription`:

```
return LaunchDescription([
    gazebo,
    spawn_robot,
    ros_gz_bridge,
    robot_state_publisher,
    joint_state_broadcaster_spawner,
    velocity_controller_spawner,
    rviz_node, # <-- Agregamos RViz aquí
])
```

Referencias e Hipervínculos:

GitHub a repositório de [runtimerobotics](#)